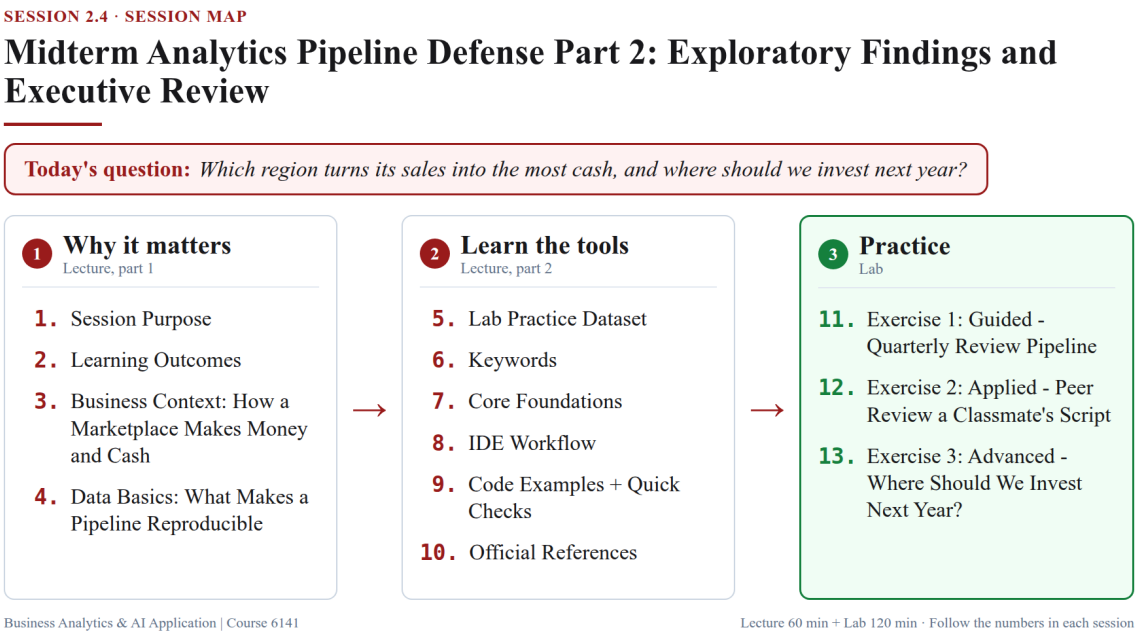
# Session 2.4: Midterm Analytics Pipeline Defense Part 2: Exploratory Findings and Executive Review

**Course 6141: Business Analytics & AI Application | Reading for Week 8**




**Schedule:** Week 8 | In-person classes (1:2 Delivery Ratio: 60 Min Lecture + 120 Min Lab)

## 1. Session Purpose

- An online marketplace earns a fee on every sale its sellers make, in three regions: North America, Europe, and Asia Pacific
- Big sales are not the same as cash in the bank: the region that sells the most may not be the one that generates the most cash
- The CFO (Chief Financial Officer) asks: which region turns its sales into the most cash, and where should we invest next year? Today, you answer with one script that anyone can rerun, and present your midterm findings in the same way

## 2. Expected Learning Outcomes (By the end of this session, you will be able to)

1. "Can someone else get the same numbers?": build a reproducible pipeline (one script that goes from the raw file to the final report with no manual steps), organized into functions

2. “Which region turns sales into the most cash?”: calculate revenue and the operating cash flow margin by region and quarter
3. “How do I explain it to the CFO?”: write findings as finding, evidence, and recommendation; review a classmate’s script; and fix a `TypeError` (mixing text and numbers)

## 3. Business Context: How a Marketplace Makes Money and Cash

- Four terms, one example (North America, 2024-Q1):

| Term | Meaning | Example |
|---|---|---|
| GMV (Gross Merchandise Value) | the total value of everything sold on the platform | `145.2` USD M |
| Take rate | the share of GMV the platform keeps as its fee | `12.5`% |
| Revenue | GMV x take rate / 100: the platform’s own income | `18.15` USD M |
| OCF (Operating Cash Flow) | the cash the business actually generated after paying its operating costs | `5.6` USD M |

- OCF margin = OCF / revenue x 100. It answers “of every dollar of revenue, how many cents became cash?” Example: `5.6 / 18.15 x 100 = 30.9`%.
- **Midterm Project, Step 2 of 2: present your findings.** In a short presentation, walk through:
    1. Your Question Card from Session 2.3 (worry, question, hypothesis, data, baseline)
    2. Your audit result: what you checked and what you fixed
    3. Your findings, each written in three parts:
        - **Finding:** one sentence a manager can repeat. “North America turns the most revenue into cash.”
        - **Evidence:** the number behind it. “Its OCF margin is 31.0%, against 24.7% in Europe and 18.5% in Asia Pacific.”
        - **Recommendation:** what to do. “Study North America’s cost structure before expanding in Asia Pacific.”
    4. A live run of your script, from the raw file to the final report

## 4. Data Basics: What Makes a Pipeline Reproducible

- A reproducible pipeline runs from top to bottom and produces the same numbers on any computer. It has four stages:

| Stage | What it does | Example in today's script |
|---|---|---|
| Load | Read the raw file | `pd.read_csv("data/session_02_04_dataset.csv")` |
| Check | Stop if the data breaks a rule | `assert (df["Gross_Merchandise_Value_USD_M"] > 0).all()` |
| Calculate | Add the measures | revenue and OCF margin |
| Report | Print and save the result | `summary.to_csv("data/region_summary.csv")` |

- Three habits that make a script reproducible:
  - Use relative paths such as `data/...`, never paths that only exist on your computer, such as `C:/Users/Reacher/Desktop/...`
  - Calculate every number from the data; never type a result into the code by hand
  - Before presenting, delete your output files and run the script again from the start

## 5. Lab Practice Dataset

- File Link: `data/session_02_04_dataset.csv`
- Audited 12-row quarterly marketplace dataset (fictional teaching data): 3 regions x 4 quarters, one row per region and quarter:
  - `Quarter_Code`: Reporting quarter, `2024-Q1` through `2024-Q4`.
  - `Market_Segment`: Region: `North_America`, `Europe`, or `Asia_Pacific`.
  - `Gross_Merchandise_Value_USD_M`: GMV (Gross Merchandise Value) in millions of U.S. Dollars.
  - `Platform_Take_Rate_Pct`: The platform's fee as a percentage of GMV.
  - `Operating_Cash_Flow_USD_M`: OCF (Operating Cash Flow) in millions of U.S. Dollars.

## 6. Keywords (Core Programming Concepts)

- `Reproducible Pipeline`:
  - What: one script that goes from the raw file to the final report, with no manual steps
  - Why: anyone can rerun it and check your numbers
- `Function` (`def` / `return`):
  - What: a named block of code that takes inputs and gives back a result
  - Why: each pipeline stage gets a clear name, and you can reuse it
- `Relative Path`:
  - What: a file path that starts from the project folder, such as `data/session_02_04_dataset.csv`
  - Why: works on every computer that has the project folder
- `Data Type` and `TypeError`:

  - What: text (`str`) and numbers (`float`, `int`) are different types; `"Total: " + 204.3` raises a `TypeError`
  - Why: printing a result is where this mistake happens most

- `Executive Finding`:
  - What: a finding, its evidence, and a recommendation
  - Why: managers act on clear statements backed by numbers, not on code

- `Peer Review`:
  - What: a classmate checks your script against a short checklist
  - Why: a second reader catches mistakes the author no longer sees

## 7. Core Programming & Computational Foundations

- Functions:
  - `def add_metrics(df):` starts a function; the indented lines below it are its body
  - `return df` hands the result back to the caller
  - Call it with `df = add_metrics(df)`. One function per pipeline stage keeps the script easy to follow

- Docstrings:
  - A text in triple quotes on the first line of a function, such as `"""Add revenue and OCF margin. """`, says what the function does
  - Reviewers read it first; `help(add_metrics)` prints it

- Mixing text and numbers:
  - `"Total: " + 204.3` raises `TypeError: can only concatenate str (not "float") to str`
  - Fix it with an f-string: `f"Total: {204.3}"`, or convert the number first: `"Total: " + str(204.3)`

- Hard-coded numbers:
  - Typing `0.125` for the take rate assumes every row has the same fee; reading the column gives each row its own fee
  - A hard-coded number may give a believable result that is still wrong, which is easy to miss

## 8. Development Workflow in Google Antigravity IDE

- `[1]` File Explorer: create your script with your name and the session code (for example, `Reacher_S24_demo_1.py`), and check the dataset is in `data/`.
- `[2]` Code Editor: write a few lines, run, repeat. Small steps keep errors small.
- `[3]` AI Agent Chat: on an error, capture the traceback (Python's error report) with `Win + Shift + S` (macOS: `Cmd + Shift + 4`), paste it with `Ctrl + V`, and ask why it failed.
- `[4]` Terminal: run `python` followed by your script name. Return code `0` means the script finished with no errors.

## 9. Hands-On Coding Micro-Examples

- Every example has three parts: **1. Input**, **2. Process**, and **3. Output**. Name each file with your own name and the session code, for example `Reacher_S24_demo_1.py` (pattern: `YourName_S24_demo_1.py`).
- *Micro-Example 1 (From GMV to Cash, Row by Row):* `YourName_S24_demo_1.py`

```
# 1. Input: load the quarterly results
import pandas as pd
df = pd.read_csv("data/session_02_04_dataset.csv")

# 2. Process: revenue, then OCF margin, for every row
df["Revenue_USD_M"] = (
    df["Gross_Merchandise_Value_USD_M"] * df["Platform_Take_Rate_Pct"] / 100
).round(2)
df["OCF_Margin_Pct"] = (
    df["Operating_Cash_Flow_USD_M"] / df["Revenue_USD_M"] * 100
).round(1)

# 3. Output: the 3 rows with the lowest margin
lowest = df.sort_values("OCF_Margin_Pct").head(3)
print(lowest[["Quarter_Code", "Market_Segment", "OCF_Margin_Pct"]])
```

  - Each row gets its own take rate from the column, so no fee is typed into the code.
  - Result: the 3 lowest margins are all Asia Pacific: `15.0` (Q1), `16.8` (Q2), and `18.6` (Q3).
- *Quick Check 1 (`.pivot_table()`):*
  - New tool: `df.pivot_table(index="A", columns="B", values="C")` makes a grid with one row per value of `A` and one column per value of `B`, like an Excel pivot table. Why: shows all 12 margins in one readable grid.
  - Task: add one line to part **3. Output**: `print(df._____(index="Market_Segment", columns="Quarter_Code", values="OCF_Margin_Pct"))`
  - Check: the `Asia_Pacific` row reads `15.0`, `16.8`, `18.6`, `22.0`: its margin improved every quarter.
- *Micro-Example 2 (The Same Work as a Pipeline of Functions):* `YourName_S24_demo_2.py`

```
# 1. Input: a function that loads and checks the raw file
import pandas as pd

def load_data(path):
    df = pd.read_csv(path)
    assert (df["Gross_Merchandise_Value_USD_M"] > 0).all(), "GMV must be above 0"
    return df

# 2. Process: functions that add measures and summarize by region
def add_metrics(df):
    df["Revenue_USD_M"] = (
```

```
        df["Gross_Merchandise_Value_USD_M"] * df["Platform_Take_Rate_Pct"] / 100
    ).round(2)
    return df

def summarize(df):
    summary = df.groupby("Market_Segment").agg(
        Revenue_USD_M=("Revenue_USD_M", "sum"),
        OCF_USD_M=("Operating_Cash_Flow_USD_M", "sum"),
    )
    summary["OCF_Margin_Pct"] = (
        summary["OCF_USD_M"] / summary["Revenue_USD_M"] * 100
    ).round(1)
    return summary

# 3. Output: run the stages in order, then print and save
df = load_data("data/session_02_04_dataset.csv")
df = add_metrics(df)
summary = summarize(df)
print(summary)
summary.to_csv("data/region_summary.csv")
```

  - Each function does one stage, and the last 5 lines show the whole pipeline at a glance.
  - Result: Asia Pacific has the most revenue (`70.39`) but the lowest margin (`18.5`%); North America has the least revenue (`65.57`) but the highest margin (`31.0`%).

- *Quick Check 2 (Docstring):*

  - New tool: a docstring, a text in triple quotes on the first line inside a function. `help(function_name)` prints it. Why: a reviewer learns what each stage does without reading its code.
  - Task: add one line directly under `def add_metrics(df):`, indented like the line below it: `"""Add revenue in USD M from GMV and the take rate."""`. Then add `help(add_metrics)` to part **3. Output**.
  - Check: the Terminal prints a help page that includes `Add revenue in USD M from GMV and the take rate.`

- *Micro-Example 3 (Write the Findings for the CFO):* `YourName_S24_demo_3.py`

```
# 1. Input: load the data and build the region summary
import pandas as pd
df = pd.read_csv("data/session_02_04_dataset.csv")
df["Revenue_USD_M"] = (
    df["Gross_Merchandise_Value_USD_M"] * df["Platform_Take_Rate_Pct"] / 100
).round(2)
summary = df.groupby("Market_Segment").agg(
    Revenue_USD_M=("Revenue_USD_M", "sum"),
    OCF_USD_M=("Operating_Cash_Flow_USD_M", "sum"),
)
```

```
# 2. Process: margin, then the best region by margin
summary["OCF_Margin_Pct"] = (
    summary["OCF_USD_M"] / summary["Revenue_USD_M"] * 100
).round(1)
ranked = summary.sort_values("OCF_Margin_Pct", ascending=False)
best_region = ranked.index[0]
best_margin = ranked["OCF_Margin_Pct"].iloc[0]

# 3. Output: finding, evidence, recommendation
print(f"Finding: {best_region} turns the most revenue into cash.")
print(f"Evidence: its OCF margin is {best_margin}%.")
print("Recommendation: study its cost structure before expanding elsewhere.")
```

  - After `groupby()`, the region names are the row labels; `ranked.index[0]` reads the first label.
  - `.iloc[0]` reads the first value of a column by position.

- *Quick Check 3 (`.idxmax()`):*
  - New tool: `column.idxmax()` returns the row label of the largest value, in one step. Why: finds "which region?" without sorting.
  - Task: add one line to part **3. Output**: `print("Most revenue:", summary["Revenue_USD_M"].______())`
  - Check: the Terminal shows `Most revenue: Asia_Pacific`. The region with the most revenue is not the one with the best margin.

## 10. Official Python Documentation & References

- Python Tutorial: Defining Functions.
- PEP 257: Docstring Conventions.
- Pandas API Reference: `DataFrame.pivot_table`.
- Cookiecutter Data Science: A Logical, Reasonably Standardized Project Structure.

## 11. Exercise 1: Guided Practice - Quarterly Review Pipeline

- *Step 1: Open Script:* In Google Antigravity IDE, create a new script named `YourName_S24_defense.py`, replacing `YourName` with your own name (for example, `Reacher_S24_defense.py`).
- *Step 2: Build the Quarterly Review:* Type the following code:

```
# 1. Input: load the quarterly results
import pandas as pd
df = pd.read_csv("data/session_02_04_dataset.csv")

# 2. Process: revenue per row, then totals per quarter
df["Revenue_USD_M"] = (
    df["Gross_Merchandise_Value_USD_M"] * df["Platform_Take_Rate_Pct"] / 100
).round(2)
```

```
quarterly = df.groupby("Quarter_Code").agg(
    GMV_USD_M=("Gross_Merchandise_Value_USD_M", "sum"),
    Revenue_USD_M=("Revenue_USD_M", "sum"),
    OCF_USD_M=("Operating_Cash_Flow_USD_M", "sum"),
)
quarterly["OCF_Margin_Pct"] = (
    quarterly["OCF_USD_M"] / quarterly["Revenue_USD_M"] * 100
).round(1)

# 3. Output: one row per quarter
print("Quarterly review:\n", quarterly)
```

- *Step 3: Write Two Findings:* Under the code, write two comment lines in the three-part form (finding, evidence, recommendation): one about the quarter with the highest GMV, and one about how the total OCF margin changed during the year.
- *Step 4: Execute in Terminal:* Run `python YourName_S24_defense.py` in Panel `[4]` and confirm return code `0`.
- *Success Check:* `2024-Q4` has the highest GMV (`524.4`) and OCF (`17.9`); Q2 has the lowest GMV (`312.4`). The total OCF margin is `23.7`% in Q1, `23.7`% in Q2, `24.6`% in Q3, and `25.8`% in Q4.

## 12. Exercise 2: Applied Practice - Peer Review a Classmate's Script

- *Coding Challenge:* A classmate sends you this script for their midterm. Copy it into `YourName_S24_practice.py`, review it, and fix it.

```
# Classmate's script: this script has mistakes on purpose
import pandas as pd
df = pd.read_csv("data/session_02_04_dataset.csv")
df["Revenue_USD_M"] = df["Gross_Merchandise_Value_USD_M"] * 0.125
total_revenue = df["Revenue_USD_M"].sum()
print("Total revenue (USD M): " + total_revenue)
```

- *Task 1 (Run It & AI Diagnosis):* Run the script. It stops with `TypeError: can only concatenate str (not "numpy.float64") to str`. Capture the traceback with `Win + Shift + S`, paste it into Panel `[3]` (AI Agent Chat) with `Ctrl + V`, and ask why text and a number cannot be joined with `+`. (`numpy.float64` is the number type that pandas returns; it behaves like a normal `float`.)
- *Task 2 (Fix the TypeError):* Change the last line to an f-string, rounding to 2 decimals: `f"Total revenue (USD M): {total_revenue:.2f}"`.
- *Task 3 (Apply the Peer Review Checklist):* Check the script against these five questions, and fix every "no":

  1. Does it run from top to bottom without errors?
  2. Does it use a relative path (`data/...`)?
  3. Is every number calculated from the data, not typed in by hand?
  4. Are outputs labeled with units?
  5. Would a manager understand the printed result?

- *Success Check:* the hard-coded `0.125` fails question 3, because each row has its own take rate. With `df["Platform_Take_Rate_Pct"] / 100`, the script prints `Total revenue (USD M): 204.31`. The hard-coded version prints `206.93`: a believable number that is wrong.

## 13. Exercise 3: Advanced Practice - Where Should We Invest Next Year?

- *Goal:* Build the region report the CFO needs for next year's budget: size, cash efficiency, and trend for every region, plus a list of weak quarters.
- *What you need:*
  - File: `data/session_02_04_dataset.csv` (described in **5. Lab Practice Dataset**).
  - Script name: `YourName_S24_challenge.py`.
  - Tools you already know: functions with `def` / `return`, `groupby()` + `.agg()`, `.pivot_table()`, `.loc[]`, `.sort_values()`, f-strings, and `.to_csv()`.
- *Requirements (follow them in order):*
  1. Write `load_data(path)` that reads the file and checks with `assert` that GMV, take rate, and OCF are all above 0.
  2. Write `add_metrics(df)` that adds `Revenue_USD_M` (2 decimals) and `OCF_Margin_Pct` (1 decimal) to every row.
  3. Write `region_report(df)` that returns one row per region with `GMV_USD_M`, `Revenue_USD_M`, `OCF_USD_M` (sums), `OCF_Margin_Pct` (from the sums, 1 decimal), and `Margin_Change_Pts`: the region's Q4 margin minus its Q1 margin, in percentage points, 1 decimal.
  4. Write `weak_quarters(df, limit)` that returns the rows with `OCF_Margin_Pct` below `limit`, sorted from lowest to highest. Call it with `limit=20`.
  5. Sort the region report by `OCF_Margin_Pct`, highest first. Print it, print the weak quarters, and save the report to `data/region_investment_report.csv`.
  6. Print one recommendation line built with an f-string from the report's numbers (not typed in by hand).
- *New tools for this exercise:*
  - Percentage points: the difference between two percentages. A margin moving from `15.0`% to `22.0`% rose by `7.0` percentage points (pts), not by 7%.
  - Reading one cell of a pivot table: `margins = df.pivot_table(index="Market_Segment", columns="Quarter_Code", values="OCF_Margin_Pct")`, then `margins["2024-Q4"] - margins["2024-Q1"]` gives the change for every region at once.
  - Adding a column that lines up by row label: `report["Margin_Change_Pts"] = (margins["2024-Q4"] - margins["2024-Q1"]).round(1)` works because both use the region names as row labels.
  - A function parameter with a value: `weak_quarters(df, limit=20)` passes `20` into `limit`.
- *Hints:*
  - Build and print each function's result before you write the next function.
  - The margin of a region must come from its total OCF divided by its total revenue, not from averaging the 4 quarterly margins.
- *Self-Check (expected results):*

– Report order: `North_America` (margin `31.0`, change `+0.2` pts), `Europe` (`24.7`, `-1.1` pts), `Asia_Pacific` (`18.5`, `+7.0` pts).
– Weak quarters (margin below 20%): `Asia_Pacific` in `2024-Q1` (`15.0`), `2024-Q2` (`16.8`), and `2024-Q3` (`18.6`).
– Business meaning: Asia Pacific is the biggest region by GMV (`563.9`) and revenue, but the least cash-efficient, and also the fastest-improving (`+7.0` pts). North America is the cash engine. A sound recommendation: keep investing in Asia Pacific's growth, tied to a target such as reaching North America's cost structure, and protect North America's margin.